בוט ימי עיון – מכללת מישלב | אפיון בנייה

GY Software · גרסה 5 · 07.11.2026

איך המסמך בנוי: לפי סדר העבודה בפועל.
חלק 1 – תשתית הבדיקות שבונים קודם. **חלק 2** – שלושת הספקים שבודקים. **חלק 3** – איך בודקים כל ספק ואיך מחליטים. **חלק 4** – הבנייה לייצור: קודם מה משמותף לכל ספק, ואחר כך ענף נפרד לכל ספק שעשוי לנצח.
למה בסדר הזה: הסיכון הכי גדול בפרויקט הוא שאף ספק לא יבין עברית טוב בטלפון. לכן בודקים אותו ראשון, על תשתית קטנה, ולא אחרי שבנינו את כל המערכת. במקביל לבדיקה אפשר כבר לבנות את החלק המשותף (4.1–4.9), כי הוא לא תלוי בספק.

תמונה כללית

1. **חלק 1 – תשתית בדיקות** (משותפת לכל הספקים)
2. **חלק 3 – בדיקת שלושת הספקים** ← החלטה על ספק
3. **חלק 4 – בנייה משותפת** (4.1–4.9) – מתחילה במקביל לבדיקה, כי היא לא תלויה בספק
4. **חלק 4 – הענף של הספק שנבחר:** ענף א (ElevenLabs Agents) / ענף ב (LiveKit + Soniox + ElevenLabs) / ענף ג (LiveKit + Gemini Live)

	שעות (הערכה)
חלק 1 – תשתית בדיקות	כ-5
חלק 3 – הקמת הספקים לבדיקה ו-18 שיחות	כ-10
חלק 4 – בנייה משותפת לכל ענף	כ-43
חלק 4 – ענף א / ב / ג	כ-15 / 21 / 19
סה"כ לפי הענף שינצח	**א: כ-73 · ב: כ-79 · ג: כ-77**

חלק 1 – תשתית הבדיקות

1.1 מה היא

בוט בדיקה מינימלי שבנוי משלושה חלקים. **החלקים 1 ו-2 זהים לכל הספקים**, ורק חלק 3 מתחלף. כך ההשוואה בין הספקים הוגנת.

#	חלק	מה הוא
1	`knowledge/` + `prompt.md`	מה הבוט יודע ואיך הוא מדבר
2	שרת כלים (Node/TS)	5 endpoints, תשובה קבועה + לוג
3	חיבור לספק	כל ספק מתחבר לחלקים 1 ו-2

שום דבר כאן לא נזרק. הפרומפט ומאגר הידע הם אלה של הייצור, ושרת הכלים הופך ל-Tools API האמיתי (4.1).

1.2 מבנה הריפו (התחלתי)

```
mishlav-bots/
  apps/
    backend/        TypeScript: tools server (mock -> real later)
    voice-worker/   Python: LiveKit agent (branches B, C only)
  packages/
    shared/         types, outcome enum, schemas
  config/
    settings.yaml   hours, attempts, retry, field/outcome mapping
    prompt.md       the agent prompt (same for all providers)
    knowledge/      study-day knowledge base
  tests/
    scenarios/      6 call scenarios + adversarial + expected results
  scripts/
    score-calls.ts  scores calls from the tool log
```

1.3 הפרומפט ומאגר הידע

- **`prompt.md`:** זרימת השיחה (4.5), הכללים העסקיים (4.6) והגבולות (4.2). אותו קובץ לכל הספקים.
- **`knowledge/`:** 2–3 ימי עיון אמיתיים מהחומרים של מישלב: נושא, תאריך, שעות, מקום, מחיר (370 ₪ בזום, 790 ₪ במלון, כולל מע"מ), מדיניות ביטול, ההטבה לשני ימים ומעלה, ושיום עיון אחד הוא 50% מדרישות ההשתלמות של לשכת רואי החשבון.

1.4 שרת הכלים (גרסת בדיקה)

5 endpoints. בבדיקה כל אחד מחזיר תשובה קבועה, **ורושם ללוג כל קריאה עם הפרמטרים והזמן.** מהלוג הזה בודקים אם הבוט הבין נכון.

כלי	מקבל	מחזיר בבדיקה
`get_day_info`	`day_id`	פרטי היום ממאגר הידע
`send_purchase_link`	`day_id`	`"נשלח"` (בלי SMS אמיתי)
`schedule_callback`	מועד מבוקש	המועד, או מועד חלופי אם הוא מחוץ לשעות הפעילות
`request_human`	`reason` מרשימה סגורה	`"הועבר לנציג"`
`end_call`	–	–

- רץ מקומית ונחשף לספקים דרך **ngrok**.
- מאובטח כבר מהבדיקה ב-secret משותף בכותרת הבקשה.

1.5 גיליון התוצאות

שורה לכל שיחה: ספק, תרחיש, שחקן, הכלי שהופעל ופרמטרים (מהלוג), תוצאה נכונה?, מעבר שפה?, טלפון נכון ספרה-ספרה?, זמן תגובה ממוצע ומקסימלי, זמן עיבוד אצל הספק, עלות השיחה. `score-calls` ממלא את רוב העמודות אוטומטית מהלוג.

1.6 שעות

ריפו ומבנה (1), פרומפט ומאגר ידע (2), שרת כלים עם לוג ו-(2) ngrok – **כ-5 שעות.**

חלק 2 – הספקים

א: ElevenLabs Agents

פלטפורמה מלאה: זיהוי דיבור (Scribe), מודל שפה, קול (v4 Turbo) וטלפוניה. אין קוד קול משלנו.

- **חזק:** הכי מהיר להעלות. הקול הכי מוכח בעברית.
- **חלש:** אין הוכחה מייצור בישראל לזיהוי עברית בטלפון. תלות מלאה בספק אחד.
- **חשבונות:** ElevenLabs, Twilio.

ב: LiveKit + Soniox + ElevenLabs

Soniox מזהה דיבור, מודל שפה מנסח, ElevenLabs מקריא. LiveKit מנהל את השיחה.

- **חזק:** ההוכחה הכי חזקה לזיהוי עברית (חברות ישראליות בייצור). כל רכיב ניתן להחלפה.
- **חלש:** יותר רכיבים לתחזק, וזמן התגובה מצטבר משלושה שלבים.
- **חשבונות:** LiveKit Cloud, Soniox, ElevenLabs (מפתח משלנו – LiveKit כבר לא מספקים את ElevenLabs בעצמם), ומפתח למודל השפה.

ג: LiveKit + Gemini Live

מודל אחד ששומע אודיו ועונה באודיו. הומלץ על ידי מפתח שבנה בוטים קוליים.

- **חזק:** זמן תגובה נמוך וזרימה טבעית. קטיעה וכלים מובנים.
- **חלש:** דיווח על גרסה 3.8 שלא מזהה "כן" ומחליפה שפה. נעילת שפה רק בפרומפט. בלי שכפול קול.
- **חשבונות:** LiveKit Cloud. מפתח Google לא חובה – אפשר להריץ דרך LiveKit Inference.

מה LiveKit עושה

Soniox ו-Gemini הם רק מודלים. הם לא עונים לטלפון. LiveKit מחבר ביניהם לבין שיחת הטלפון: מקבל או יוצר את השיחה (SIP), מזרים את האודיו, מזהה מתי הלקוח סיים לדבר, עוצר את הבוט כשקוטעים אותו, ומפעיל את הכלים שלנו. ב-ElevenLabs Agents כל זה כבר בפנים, ולכן בענף א אין LiveKit.

חלק 3 – איך בודקים כל ספק

3.1 הקמה – ספק א (כ-2 שעות)

1. פותחים סוכן בדשבורד של ElevenLabs, מדביקים את `prompt.md` ומעלים את מאגר הידע.
2. מגדירים את 5 הכלים כ-webhooks לכתובת ה-ngrok של שרת הכלים.
3. מחברים מספר Twilio לסוכן, ומאמתים בחשבון Twilio את הניידים של הצוות.
4. **הבוט מתקשר אל הצוות** (כפתור Outbound call בדשבורד, או ה-API).

> **לבדוק ראשון:** חשבון ניסיון של Twilio כולל כ-75 דקות שיחה לחודש הראשון ומתקשר רק למספרים מאומתים. לפי מרכז העזרה של ElevenLabs, החיבור ל-Twilio דורש חשבון Twilio בתשלום. סביר שנצטרך לשדרג (עשרות דולרים).

3.2 הקמה – ספקים ב ו-ג (כ-4 שעות)

1. סוכן Python אחד ב-LiveKit, לפי ה-Voice AI Quickstart, עם מתג: `VOICE_PROVIDER=soniox_eleven | gemini`.

2. אותו `prompt.md`, ואותם 5 כלים שקוראים לאותו שרת כלים.
3. שוכרים את המספר האמריקאי החינמי של LiveKit ומשייכים אותו לסוכן.
4. **הצוות מתקשר אל הבוט** – המספר של LiveKit מקבל כרגע רק שיחות נכנסות.

> **לשים לב:** שיחה מהנייד למספר בארה"ב עשויה לעלות כסף לפי החבילה. תוכנית Build כוללת 50 דקות נכנסות חינם.

3.3 התרחישים (6 לכל ספק, 18 בסך הכול)

שני אנשי צוות (גבר ואישה) משחקים לקוח, וחלק מהשיחות מהנייד מחוץ למשרד.

1. **רוצה להירשם:** "כן", שואל על מחיר ומיקום, מבקש קישור. צפוי: `LINK_SENT`.
2. **עסוק:** "תחזרי אליי ביום שלישי הבא בעשר". צפוי: `CALLBACK` עם המועד הנכון.
3. **טלפון ושם:** מבקש נציג ומכתיב מספר ושם. צפוי: `HOT_LEAD` עם הפרטים הנכונים.
4. **קטיעה ורעש:** רעש רחוב, קוטע את הבוט ומדבר מהר. צפוי: הבוט עוצר וממשיך נכון.
5. **הסרה עם אנגלית:** "יש לי zoom עכשיו, ותורידי אותי מהרשימה". צפוי: `DO_NOT_CALL`, בלי מעבר לאנגלית.
6. **משיבון:** צפוי: `VOICEMAIL` וניתוק בלי הודעה.

3.4 מה מודדים

- **הבנה:** הכלי הנכון עם הפרמטרים הנכונים (מהלוג, לא מהתמלול – כך גם Gemini נבדק בהוגנות).
- **שפה:** עברית לאורך כל השיחה.
- **זמן תגובה:** מסוף דיבור הלקוח עד תחילת התשובה. **בנפרד:** זמן העיבוד אצל הספק, כדי לא לפסול ספק בגלל הנתיב דרך ארה"ב.
- **קול:** 1–5 משלושה מאזינים, בלי לדעת איזה ספק (קטעי 10 שניות מסומנים X/Y/Z).
- **עלות לדקה:** כולל טלפוניה ומודל שפה, מהדשבורד של כל ספק.

3.5 תנאי מעבר והחלטה

1. הבנה נכונה בכל 6 התרחישים.
2. אפס מעברי שפה.
3. מספר הטלפון נכון ספרה-ספרה.
4. זמן תגובה ממוצע עד 1.2 שניות, ואף תגובה מעל 2 שניות.
5. ציון קול ממוצע 4 ומעלה.

> **החלטה:** מי שעובר את כל התנאים. אם כמה עוברים – א, אחריו ג, אחריו ב (לפי פשטות תחזוקה), ועלות לדקה מכריעה במקרה של קרבה. נכשל רק בזמן תגובה? קודם מכוונים, ורק אז פוסלים. **אף ספק לא עבר?** עוצרים ומעדכנים את מישלב לפני שממשיכים.

3.6 שעות

הקמת א (2), הקמת ב ו-ג (4), 18 שיחות, גיליון, דירוג והחלטה (4) – **כ-10 שעות.**

3.7 לינקים

- **חשבונות:** ElevenLabs – מפתחות · Soniox – הרשמה · Google AI Studio – מפתח · Twilio – חשבון ניסיון · Twilio – מגבלות ניסיון · Twilio – אימות מספרים
- **ספק א:** חיבור Twilio לסוכן · כלים · שיחה יוצאת – CLI · API
- **ספקים ב ו-ג:** Gemini Live · ElevenLabs TTS · Soniox STT · Voice AI Quickstart · מספרי טלפון · טלפוניה

- **עזר:** ngrok

חלק 4 – הבנייה לייצור

חלק משותף לכל הענפים

4.1 רכיבי המערכת

- **Importer:** קליטת האקסל מגולד, ולידציה, נרמול טלפונים, מניעת כפילויות, חילוץ שדות מההערות (4.3), טעינה ל-Supabase. מיפוי עמודות בקובץ הגדרות.
- **Supabase:** מקור האמת – אנשי קשר, תור חיוג, ניסיונות שיחה, תוצאות, טבלת חסימה, ייבוא וייצוא, יומן פעולות. פרויקט נפרד ל-dev / test / prod.
- **Dialer:** בחירה לפי כללי החיוג (4.7), נעילה אטומית, עד 20 שיחות במקביל (בכפוף למכסות הספק, עם backoff). לפני כל חיוג: בדיקת טבלת החסימה ושמירת `call_id → contact_id`.
- **Tools API:** שרת הכלים מחלק 1, עם לוגיקה אמיתית במקום תשובות קבועות.
- **Export Feed:** כל התוצאות שהשתנו מאז המשיכה האחרונה של גולד, מאובטח במפתח API.
- **Notifier:** התראה מיידית לנציג על ליד חם (וואטסאפ או מייל).
- **ממשק `VoiceProvider`:** שכבת הקול מאחורי ממשק אחד. כל ענף מממש אותו. בנוסף `MockProvider` לבדיקות.

```
interface VoiceProvider {
  startCall(contact: Contact, ctx: CallContext): Promise<{ callId: string }>;
  onCallEnded(handler: (r: CallResult) => Promise<void>): void;
}
type CallResult = {
  callId: string; outcome: Outcome; reason?: string; summary: string;
  callbackAt?: string; durationSec: number; turns: TurnMetric[];
};
```

פריסה: Backend על Render (אזור פרנקפורט) – שירות web ל-Tools API ול-Export, ו-background worker לחייגן. אין צורך ב-IP קבוע.

4.2 ארכיטקטורת הסוכן

חלק	ארכיטקטורה	נימוק
השיחה עצמה	**ReAct מוגבל** (הלולאה רצה אצל הספק)	כל משפט של הלקוח משנה את הצעד הבא, והלקוח אמור לנווט את השיחה
כל מה שמסביב	**קוד רגיל, בלי LLM**	את השלבים אנחנו יודעים מראש

- **לא Plan-then-Execute:** אי אפשר לתכנן שיחה מראש, ושלב תכנון מוסיף זמן תגובה. העיקרון שלו – מידע ממקור שלישי לא מנווט את הסוכן – מיושם בהערות (4.3).
- **לא Multi-Agent:** סוכן נפרד לכל בוט, בלי תזמור ביניהם.
- **"מוגבל":** 5 כלים, קריאה אחת לכלי בכל תור, ואף כלי שיכול לגרום נזק.
- **שני גבולות:** הפרומפט קובע מה הבוט **אומר** (אפשר לעקוף). הכלים והקוד קובעים מה הוא **יכול לעשות** (אי אפשר לעקוף). כל איסור קריטי ממומש גם בקוד. **המודל מציע, ה-Backend מחליט.**

הכלים בייצור:

כלי	מה המודל מעביר	מה השרת קובע לבד
`get_day_info`	`day_id` מרשימה סגורה	התוכן, ממאגר הידע בלבד
`send_purchase_link`	`day_id` מרשימה סגורה	הקישור (קבוע לכל יום), הטלפון (מה-DB), שהיום פתוח להרשמה
`schedule_callback`	מועד מבוקש	שהמועד עתידי ובשעות הפעילות, אחרת מועד חלופי
`request_human`	`reason`: `wants_human` / `pay_via_rep` / `company_purchase` / `discount_request` / `other`	יצירת משימה, סימון ליד חם, התראה
`end_call`	–	–

אין פרמטר של טלפון, מחיר, URL או `contact_id`. לבוט אין כלי כתיבה ל-DB או לגולד. אין כלי הנחה או תשלום.

4.3 אבטחה ונתונים

- **זהות בשרת:** כל קריאה לכלי וכל שמירת תוצאה עוברות דרך `call_id → contact_id`, לא לפי מה שנאמר בשיחה.
- **צמצום מידע:** ההקשר שמוזרק לשיחה (`CallContext`) כולל רק שם פרטי, יום העיון, שדות שחולצו מההערות ותוצאות קודמות מרשימה סגורה. לא נכנסים: ת.ז, טלפונים, ההערה המקורית, נתוני לקוחות אחרים.
- **הערות מגולדד:** ההערה המקורית לא מגיעה לבוט. בייבוא, קריאת LLM נפרדת מחלצת רק `needs_human`, `do_not_call`, `interested_day_id`, `preferred_call_time`, וה-Backend מאמת. אפשר לכבות את השלב אם ההערות ריקות.
- **הערות של הבוט:** כל סיכום שהבוט כותב מתחיל ב-`[BOT]`, ובייבוא מדלגים עליו – קוראים רק הערות של נציגים.
- **endpoints:** אימות secret או חתימה של הספק, ובדיקה שה-`call_id` שייך לשיחה פעילה.
- **מפתחות:** service key של Supabase רק בשרת, RLS על כל הטבלאות, סודות במשתני סביבה.
- **פרטי אשראי:** לפני שמירת תמלול או סיכום נמחק כל רצף של 13–19 ספרות. אם הספק מציע מחיקת פרטים רגישים, מפעילים גם אותה.
- **חילוץ בסוף שיחה:** ה-Backend מאמת את `CallResult` מול רשימות סגורות ותאריכים חוקיים. ערך לא תקין מסומן לבדיקה ידנית.
- **משך שיחה מקסימלי** מוגדר בסוכן.

4.4 מה לא בונים

Planner, Multi-Agent, LLM-as-judge בזמן אמת, RAG / Vector DB לכמה ימי עיון, Policy engine, אישור אנושי לכל פעולה, Fallback אוטומטי בין ספקים (רק עצירת חיוג כשהספק נופל).

4.5 זרימת השיחה

התסריט של מישלב נכתב לוואטסאפ. בשיחה קולית ללקוח קיים רוב איסוף הפרטים מיותר.

1. **פתיחה:** "שלום [שם], מדברת [שם הבוט] ממכללת מישלב. התעניינת בימי העיון לחשבי שכר, יש לך דקה?" ← לא עכשיו: `CALLBACK`.
2. **איזה יום:** לפי מה שהתעניין בו, או לפי נושא ופורמט (זום / מלון). לא מקריאים את כל התאריכים.
3. **פרטים:** תאריך, שעות, מקום ומחיר, ממאגר הידע.
4. **שאלות:** רק ממאגר הידע. לא יודע ← `request_human`.
5. **בדיקת עניין:** "לשלוח לך קישור להרשמה?"
6. **התנגדות:** ניסיון שכנוע **אחד** (50% מדרישות הלשכה; הטבה של 300 ₪ לקורס בשני ימים ומעלה). עדיין לא ← `NOT_INTERESTED` עם הסיבה.
7. **סגירה:** קונה לבד ← קישור (`LINK_SENT`). נציג לתשלום ← `HOT_LEAD`. חברה משלמת ← `COMPANY`.

טופס חברה, העברה בנקאית ורישום במערכת "מנהל" נשארים אצל הצוות. **"נשלח קישור" הוא הסוף מבחינת הבוט:** בגולד נכתב "נשלח קישור לרכישה" (לא "נרשם"), ונפתח מעקב לנציג בעוד 3 ימים.

4.6 כללים עסקיים

- **תשלום:** הבוט רק שולח קישור. לא מקבל פרטי אשראי (עוצר לקוח שמקריא), לא מאשר הרשמה.
- **הנחות:** לא נותן, לא מתמקח, לא מבטיח מחיר. כל בקשה ← `request_human(discount_request)` + ליד חם.
- **מידע:** רק ממאגר הידע. כשאין תשובה – "נציג יחזור אליך".

4.7 תוצאות וחיוג חוזר

מתקשרים שוב	מתי
`VOICEMAIL` / `BUSY` / `NO_ANSWER`	בעוד 3 שעות; אחרי סוף היום – בבוקר יום העבודה הבא
`DROPPED`	בעוד שעה, נספר כניסיון
`CALLBACK`	במועד שביקש, בתוך שעות הפעילות
`FAILED`	בעוד 15 דקות, לא נספר כניסיון
`WRONG_NUMBER`	טלפון 2, אחר כך טלפון 3, אחרת סגירה

לא מתקשרים שוב	למה
`LINK_SENT`	הבוט סיים; מעקב לנציג בעוד 3 ימים
`HOT_LEAD` / `COMPANY` / `HUMAN_FOLLOWUP`	עבר לנציג
`NOT_INTERESTED`	אחרי ניסיון שכנוע אחד
`ALREADY_REGISTERED`	סגירה
`DO_NOT_CALL`	**חסימה קבועה** לפי טלפון, נבדקת לפני כל חיוג, מסונכרנת לגולד
`closed_max_attempts`	אחרי 4 ניסיונות
סגור / "לא מעוניין" בקובץ מגולד, או לא הופיע בקובץ החדש	לא נכנס לתור

ברירות מחדל לטסטים (ב-`settings.yaml`, יוצגו למישלב לאישור אחרי שהמערכת תעבוד): 3 שעות בין ניסיונות · 4 ניסיונות · מעקב אחרי 3 ימים · א'–ה' 09:00–17:00 · בלי שישי, שבת וחגים · `Asia/Jerusalem` כולל שעון קיץ/חורף · מתג עצירה עוצר חיוג חדש, ושיחות פעילות מסתיימות כרגיל.

4.8 בדיקות וניטור

- **סביבות:** dev / test / prod. ב-dev ו-test החייגן מחייג רק לרשימת מספרי הצוות.
- **יחידה:** כללי חזרה ושעות, נרמול, כפילויות, idempotency, נעילות, חסימה, ולידציה של כלים וחילוץ.
- **אינטגרציה:** `MockProvider` ← Dialer ← Supabase ← Importer ← תוצאות ← Export, כולל השהיות, ניתוקים ומשיבון.
- **עומס:** 20 שיחות במקביל – בלי חיוג כפול ובלי רשומה תקועה.
- **בדיקות טקסט אוטומטיות:** התרחישים כטקסט מול הפרומפט והכלים, בכל commit (המימוש תלוי בענף).
- **שיחות אמיתיות:** 6 התרחישים, לפני כל פריסה, ובסוף דרך Voice Center ממספר המכללה.
- **תסריטים "רעים":** הזרקה בעברית ובאנגלית, ניסיון להוציא מידע על לקוח אחר, קישור למספר אחר, בקשת הנחה, יום עיון שלא קיים, הקראת אשראי (ולוודא שלא נשמר), התחזות.
- **ניטור:** לוג מובנה לפי `/healthz` ,Callid לכל שירות, התראה על חריגה בזמן תגובה או בשיעור כשלונות.

4.9 שעות – חלק משותף

רכיב	שעות
Supabase: סכמה, מיגרציות, RLS, סביבות	3
Importer כולל חילוץ הערות ודילוג על `[BOT]`	7
Dialer: תור, נעילות, חסימה, כללי חזרה, שעות, חגים	9
Tools API אמיתי: לוגיקה, אימות, ולידציה, מחיקת אשראי	6
Export Feed	4
Notifier	3
MockProvider	2
בדיקות יחידה, אינטגרציה ועומס	7
פריסה ל-Render, ניטור ומתג עצירה	2
סה"כ	**43**

ענף א – ElevenLabs Agents

מה מיוחד בענף: אין קוד קול משלנו ואין `voice-worker`. הסוכן חי בפלטפורמה של ElevenLabs, והקוד שלנו רק מפעיל שיחות ומקבל תוצאות.

איך שיחה רצה:

1. החייגן קורא ל-API של שיחה יוצאת של ElevenLabs עם הסוכן, מספר היעד וה-`CallContext` (משתנים דינמיים).
2. ElevenLabs מחייגים דרך Voice Center (SIP trunk) ממספר המכללה.
3. במהלך השיחה הסוכן קורא לכלים כ-webhooks ל-Tools API.
4. בסוף השיחה ElevenLabs שולחים post-call webhook עם התמלול והנתונים שחולצו ← ה-Backend מאמת ושומר.

מה בונים:

- `ElevenLabsAgentsProvider` שמממש את `VoiceProvider`: שיחה יוצאת, קליטת post-call webhook, אימות החתימה.
- **הסוכן כקוד:** הפרומפט, הכלים ושדות החילוץ מנוהלים ב-CLI של `agents push / pull` (ElevenLabs בתוך הריפו).
- **משיבון:** זיהוי משיבון של הספק (בשיחות דרך Twilio יש AMD מובנה; מול Voice Center – לבדוק) + כלל בפרומפט לנתק בלי הודעה.
- **חיבור Voice Center:** SIP trunk של Voice Center ל-ElevenLabs, ובדיקה ממספר המכללה.
- **בדיקות טקסט:** בדיקות הסוכן המובנות של ElevenLabs (סימולציה ובדיקות כלים), מורצות מה-CLI בכל commit.

שעות:

רכיב	שעות
`ElevenLabsAgentsProvider` + post-call webhook	4
פרומפט, כלים ושדות חילוץ – כוונון	6
משיבון ובדיקות טקסט	2
חיבור Voice Center ובדיקה	3
סה"כ ענף	**15**

ענף ב – LiveKit + Soniox + ElevenLabs

מה מיוחד בענף: יש `voice-worker` משלנו ב-Python שרץ על LiveKit Cloud, ובו שלושה רכיבים נפרדים: Soniox (שמיעה), מודל שפה (חשיבה), ElevenLabs (דיבור).

איך שיחה רצה:

1. החייגן קורא ל-API של LiveKit ליצירת משתתף SIP – כלומר שיחה יוצאת דרך outbound trunk של Voice Center.
2. LiveKit מכניס את ה-`voice-worker` לחדר עם ה-`CallContext`.
3. הכלים מוגדרים כפונקציות ב-worker, וכל אחת קוראת ל-Tools API.
4. בסוף השיחה ה-worker שולח `CallResult` ל-Backend ← אימות ושמירה.

מה בונים:

- `LiveKitProvider` שמממש את `VoiceProvider` (משותף לענפים ב ו-ג).
- `voice-worker` לייצור: Soniox STT עם רמזי שפה עברית, מודל שפה, ElevenLabs TTS עם הקול שנבחר, זיהוי סוף דיבור וקטיעה.
- **חילוץ בסוף שיחה:** קריאת LLM על התמלול שממלאת `CallResult`.
- **משיבון:** זיהוי בעצמנו (למשל לפי ברכת פתיחה ארוכה בלי תגובה) + ניתוק.
- **חיבור Voice Center:** outbound trunk ב-LiveKit מול Voice Center.
- **בדיקות טקסט:** הרצת הסוכן עם קלט טקסט בלי קול, מול התרחישים, בכל commit.
- **כוונון:** שלושה רכיבים = שלוש נקודות לכוונן זמן תגובה.

שעות:

רכיב	שעות
`voice-worker` + `LiveKitProvider` לייצור	6
חילוץ בסוף שיחה	2
פרומפט, כלים וכוונון זמני תגובה	6
משיבון	2
בדיקות טקסט	2
חיבור Voice Center ופריסה ל-LiveKit Cloud	3
סה"כ ענף	**21**

ענף ג – LiveKit + Gemini Live

מה מיוחד בענף: כמו ענף ב, אבל במקום שלושה רכיבים יש מודל אחד ששומע ומדבר. פחות כוונן, אבל צריך לשמור על העברית בפרומפט.

איך שיחה רצה: זהה לענף ב (שלבים 1–4).

מה בונים:

- אותו `LiveKitProvider` ואותו `voice-worker`, עם `RealtimeModel` של Gemini במקום שלושת הרכיבים.
- **נעילת עברית:** אין הגדרה, רק פרומפט – לכן כלל חד בפרומפט + בדיקת טקסט ייעודית למעבר שפה, וניטור מעברי שפה בייצור.
- **חילוץ בסוף שיחה:** קריאת LLM על התמלול, כמו בענף ב.
- **משיבון, בדיקות טקסט, חיבור Voice Center:** כמו בענף ב.

שעות:

רכיב	שעות
`voice-worker` + `LiveKitProvider` עם Gemini	5
חילוץ בסוף שיחה	2
פרומפט, כלים ונעילת שפה	5
משיבון	2
בדיקות טקסט כולל בדיקת מעבר שפה	2
חיבור Voice Center ופריסה ל-LiveKit Cloud	3
סה"כ ענף	**19**

חלוקת עבודה מוצעת

- **גיא:** חלק 1, חלק 3, והענף שינצח (הקול).
- **יאיר:** החלק המשותף 4.1–4.9 (המערכת), במקביל לבדיקת הספקים, מול `MockProvider`.
- **ביחד:** בדיקות, חיבור Voice Center ופיילוט.

פתוח מול מישלב (לא חוסם את הבנייה)

- קובץ אקסל לדוגמה מגולד – עמודות, מזהה פעילות, ומה יש בהערות.
- איך גולד מושכים את התוצאות.
- ערוץ לקישור: SMS או וואטסאפ.
- אישור ברירות המחדל (4.7) ושהרשימה מותרת לפנייה שיווקית.
- הקלטת שיחות כן/לא (אם כן – הודעה בפתיחה).